Complete GCP Setup Guide - GUI + CLI

AI-Powered Product Review Analyzer - Dual Approach

This guide provides **both GUI (Console) and CLI** instructions for every step.

Table of Contents

---

1. Project Setup

🖥️ GUI Method

Step 1: Create GCP Account

1. Go to https://cloud.google.com
2. Click **"Get started for free"**
3. Sign in with your Google account
4. Complete billing information
5. You'll receive $300 free credits

Step 2: Create New Project

1. Go to https://console.cloud.google.com
2. Click the project dropdown (top bar, next to "Google Cloud")
3. Click **"NEW PROJECT"**
4. Enter project details:
    - **Project name**: `AI Review Analyzer`

    - **Project ID**: `devfest-review-analyzer` (must be unique)
    - **Location**: Leave as default or select your organization
5. Click **"CREATE"**
6. Wait for project creation (takes ~30 seconds)
7. Select the new project from the dropdown

Screenshots to take:

- Project selection dropdown
- New project creation dialog
- Project dashboard after creation

💻 CLI Method

bash

```
# 1. Install gcloud CLI
# macOS:
curl https://sdk.cloud.google.com | bash
exec -l $SHELL

# Linux:
curl -O
https://dl.google.com/dl/cloudsdk/channels/rapid/downloads/google-cloud-cli-linux-x86_64.tar.gz
tar -xf google-cloud-cli-linux-x86_64.tar.gz
./google-cloud-sdk/install.sh

# 2. Initialize gcloud
gcloud init

# 3. Create project
export PROJECT_ID="devfest-review-analyzer"

gcloud projects create $PROJECT_ID \
    --name="AI Review Analyzer" \
    --set-as-default

# 4. Set as default
gcloud config set project $PROJECT_ID

# 5. Link billing account
# First, list billing accounts
gcloud billing accounts list

# Link billing (replace BILLING_ACCOUNT_ID with actual ID)
gcloud billing projects link $PROJECT_ID \
    --billing-account=BILLING_ACCOUNT_ID
```

```
# 6. Verify
gcloud config list
gcloud projects describe $PROJECT_ID
```

CLI Output to Expect:
```
Created
[https://cloudresourcemanager.googleapis.com/v1/projects/devfest-review-analyzer].
name: AI Review Analyzer
projectId: devfest-review-analyzer
projectNumber: '123456789012'
lifecycleState: ACTIVE
```

---

2. Enable APIs

🖥️ GUI Method

Step 1: Navigate to APIs & Services

1. In the GCP Console, open the navigation menu (☰)
2. Go to **"APIs & Services"** → **"Library"**

Step 2: Enable Each API

For each API below, search for it and click **"ENABLE"**:

1. **Cloud Run API**
   - Search: "Cloud Run API"
   - Click on "Cloud Run API"
   - Click **"ENABLE"**
   - Wait for activation (~10 seconds)
2. **Cloud Pub/Sub API**
   - Search: "Cloud Pub/Sub API"
   - Click **"ENABLE"**
3. **Cloud Firestore API**
   - Search: "Cloud Firestore API"
   - Click **"ENABLE"**
4. **Cloud Storage API**
   - Search: "Cloud Storage"
   - Click **"ENABLE"**
5. **Cloud Logging API**
   - Search: "Cloud Logging API"
   - Click **"ENABLE"**
6. **Cloud Trace API**

    - Search: "Cloud Trace API"
    - Click **"ENABLE"**
7. **Cloud Monitoring API**
    - Search: "Cloud Monitoring API"
    - Click **"ENABLE"**
8. **Cloud Build API**
    - Search: "Cloud Build API"
    - Click **"ENABLE"**
9. **Secret Manager API**
    - Search: "Secret Manager API"
    - Click **"ENABLE"**
10. **Artifact Registry API**
    - Search: "Artifact Registry API"
    - Click **"ENABLE"**
11. **Identity and Access Management (IAM) API**
    - Search: "IAM API"
    - Click **"ENABLE"**

Step 3: Verify Enabled APIs

1. Go to **"APIs & Services"** → **"Enabled APIs & Services"**
2. You should see all 11 APIs listed

Screenshots to take:

- API Library search page
- Individual API enable page
- Enabled APIs list

💻 CLI Method

bash

```
# Enable all APIs at once
gcloud services enable \
  run.googleapis.com \
  pubsub.googleapis.com \
  firestore.googleapis.com \
  storage.googleapis.com \
  logging.googleapis.com \
  cloudtrace.googleapis.com \
  monitoring.googleapis.com \
  cloudbuild.googleapis.com \
  secretmanager.googleapis.com \
  artifactregistry.googleapis.com \
  iam.googleapis.com

# This takes 2-3 minutes

# Verify all APIs are enabled
```

```
gcloud services list --enabled

# Or check specific API
gcloud services list --enabled --filter="name:run.googleapis.com"
```

CLI Output:
```
NAME                              TITLE
run.googleapis.com                Cloud Run API
pubsub.googleapis.com             Cloud Pub/Sub API
firestore.googleapis.com          Cloud Firestore API
storage.googleapis.com            Cloud Storage API

...
```

---

3. Service Account Creation

🖥️ GUI Method

Step 1: Navigate to Service Accounts

1. Open navigation menu (☰)
2. Go to **"IAM & Admin"** → **"Service Accounts"**
3. Click **"+ CREATE SERVICE ACCOUNT"**

Step 2: Create Service Account

1. **Service account details:**
   - **Service account name**: `review-analyzer-sa`
   - **Service account ID**: `review-analyzer-sa` (auto-filled)
   - **Description**: `Service account for all review analyzer microservices`
2. Click **"CREATE AND CONTINUE"**

Step 3: Grant Roles (Part 1 - Runtime Roles) Click **"+ ADD ANOTHER ROLE"** for each role:

1. **Cloud Run Admin**
   - Filter: "Cloud Run Admin"
   - Select: `Cloud Run Admin`
   - Click "ADD"
2. **Service Account User**
   - Filter: "Service Account User"
   - Select: `Service Account User`
3. **Storage Admin**

    - Filter: "Storage Admin"
    - Select: `Storage Admin`
4. **Pub/Sub Admin**
    - Filter: "Pub/Sub Admin"
    - Select: `Pub/Sub Admin`
5. **Cloud Datastore Owner**
    - Filter: "Cloud Datastore Owner"
    - Select: `Cloud Datastore Owner`
6. **Logging Admin**
    - Filter: "Logging Admin"
    - Select: `Logging Admin`
7. **Monitoring Admin**
    - Filter: "Monitoring Admin"
    - Select: `Monitoring Admin`
8. **Cloud Trace Admin**
    - Filter: "Cloud Trace Admin"
    - Select: `Cloud Trace Admin`
9. **Secret Manager Admin**
    - Filter: "Secret Manager Admin"
    - Select: `Secret Manager Admin`
10. **Artifact Registry Administrator**
    - Filter: "Artifact Registry Administrator"
    - Select: `Artifact Registry Administrator`

Step 4: Finalize

1. Click **"CONTINUE"**
2. Click **"DONE"**

Step 5: Create Key (for local development)

1. Find your service account in the list
2. Click the **three dots (⋮)** → **"Manage keys"**
3. Click **"ADD KEY"** → **"Create new key"**
4. Select **"JSON"**
5. Click **"CREATE"**
6. The key file downloads automatically
7. Save it as `service-account-key.json` in your project's `credentials/` folder

Screenshots to take:

- Service account creation form
- Grant access page with roles
- Keys page
- Downloaded JSON key

💻 CLI Method

bash

```bash
# 1. Set variables
export SA_NAME="review-analyzer-sa"
export SA_EMAIL="${SA_NAME}@${PROJECT_ID}.iam.gserviceaccount.com"

# 2. Create service account
gcloud iam service-accounts create $SA_NAME \
    --display-name="Review Analyzer Service Account" \
    --description="Service account for all review analyzer microservices"

# 3. Grant all required roles
ROLES=(
    "roles/run.admin"
    "roles/iam.serviceAccountUser"
    "roles/storage.admin"
    "roles/pubsub.admin"
    "roles/datastore.owner"
    "roles/logging.admin"
    "roles/monitoring.admin"
    "roles/cloudtrace.admin"
    "roles/secretmanager.admin"
    "roles/artifactregistry.admin"
)

for ROLE in "${ROLES[@]}"; do
    echo "Granting $ROLE..."
    gcloud projects add-iam-policy-binding $PROJECT_ID \
        --member="serviceAccount:${SA_EMAIL}" \
        --role="$ROLE"
done

# 4. Create key
mkdir -p credentials
gcloud iam service-accounts keys create credentials/service-account-key.json \
    --iam-account=${SA_EMAIL}

# 5. Set environment variable
export
GOOGLE_APPLICATION_CREDENTIALS="$(pwd)/credentials/service-account-key.json"

# 6. Verify
gcloud iam service-accounts list
gcloud projects get-iam-policy $PROJECT_ID \
    --flatten="bindings[].members" \
    --filter="bindings.members:serviceAccount:${SA_EMAIL}"
```

---

4. Cloud Storage Setup

🖥️ GUI Method

Step 1: Navigate to Cloud Storage

1. Open navigation menu (☰)
2. Go to **"Cloud Storage"** → **"Buckets"**
3. Click **"CREATE"**

Step 2: Create Bucket

1. **Name your bucket:**
    - **Name**: `devfest-review-analyzer-reviews-raw`
    - Must be globally unique - if taken, add numbers: `devfest-review-analyzer-reviews-raw-001`
    - Click **"CONTINUE"**
2. **Choose where to store your data:**
    - **Location type**: Region
    - **Region**: `us-central1 (Iowa)`
    - Click **"CONTINUE"**
3. **Choose a storage class:**
    - **Default class**: Standard
    - Click **"CONTINUE"**
4. **Choose how to control access:**
    - **Uncheck** "Enforce public access prevention"
    - **Select** "Uniform" (Bucket or object-level)
    - Click **"CONTINUE"**
5. **Choose how to protect object data:**
    - Leave all as default
    - Click **"CREATE"**

Step 3: Set Lifecycle Policy

1. Click on your newly created bucket
2. Go to **"LIFECYCLE"** tab
3. Click **"ADD A RULE"**

Rule 1: Delete after 90 days

1. **Select action:**
    - Check **"Delete object"**
    - Click **"CONTINUE"**
2. **Select object conditions:**
    - **Age**: 90 days
    - Click **"CONTINUE"**
3. Click **"CREATE"**

Rule 2: Move to NEARLINE after 7 days

1. Click **"ADD A RULE"** again
2. **Select action:**
    - Check **"Set storage class to Nearline"**
    - Click **"CONTINUE"**
3. **Select object conditions:**
    - **Age**: 7 days
    - Click **"CONTINUE"**
4. Click **"CREATE"**

Step 4: Grant Permissions

1. Go to **"PERMISSIONS"** tab
2. Click **"GRANT ACCESS"**
3. **Add principals:**
    - **New principals**: Paste your service account email: `review-analyzer-sa@devfest-review-analyzer.iam.gserviceaccount.com`
4. **Assign roles:**
    - Select **"Storage Object Admin"**
5. Click **"SAVE"**

Screenshots to take:

- Bucket creation form
- Lifecycle rules list
- Permissions page

💻 CLI Method

bash

```
# 1. Set bucket name
export BUCKET_NAME="${PROJECT_ID}-reviews-raw"

# 2. Create bucket
gsutil mb -l us-central1 gs://${BUCKET_NAME}

# 3. Set uniform bucket-level access
gsutil uniformbucketlevelaccess set on gs://${BUCKET_NAME}

# 4. Create lifecycle policy file
cat > lifecycle-policy.json << 'EOF'
{
  "lifecycle": {
    "rule": [
      {
        "action": {
          "type": "Delete"
```

```
      },
      "condition": {
        "age": 90
      }
    },
    {
      "action": {
        "type": "SetStorageClass",
        "storageClass": "NEARLINE"
      },
      "condition": {
        "age": 7
      }
    }
  ]
 }
}
EOF

# 5. Apply lifecycle policy
gsutil lifecycle set lifecycle-policy.json gs://${BUCKET_NAME}

# 6. Grant permissions to service account
gsutil iam ch serviceAccount:${SA_EMAIL}:roles/storage.objectAdmin
gs://${BUCKET_NAME}

# 7. Verify
gsutil ls
gsutil lifecycle get gs://${BUCKET_NAME}
gsutil iam get gs://${BUCKET_NAME}
```

---

5. Pub/Sub Configuration

🖥️ GUI Method

Step 1: Create Topic

1. Open navigation menu (☰)
2. Go to **"Pub/Sub"** → **"Topics"**
3. Click **"CREATE TOPIC"**
4. **Topic ID**: `review-submitted`
5. **Check** "Add a default subscription"
6. **Subscription ID**: `review-submitted-sub`
7. **Message retention duration**: 7 days
8. Click **"CREATE"**

Step 2: Create Dead Letter Topic

1. Click **"CREATE TOPIC"** again
2. **Topic ID**: `review-submitted-dead-letter`
3. **Uncheck** "Add a default subscription"
4. Click **"CREATE"**

Step 3: Configure Subscription (we'll update this later after deploying AI service)

1. Go to **"Subscriptions"** tab
2. Click on `review-submitted-sub`
3. Click **"EDIT"**
4. **Delivery type**: We'll change this to Push later
5. **Acknowledgement deadline**: 600 seconds
6. **Message retention duration**: 7 days
7. **Retry policy:**
    - **Minimum backoff**: 10 seconds
    - **Maximum backoff**: 600 seconds
8. Click **"UPDATE"**

Step 4: Grant Permissions

1. Click on `review-submitted` topic
2. Go to **"PERMISSIONS"** tab
3. Click **"ADD PRINCIPAL"**
4. **New principals**: `review-analyzer-sa@devfest-review-analyzer.iam.gserviceaccount.com`
5. **Role**: `Pub/Sub Publisher`
6. Click **"SAVE"**
7. Go back to **"Subscriptions"**
8. Click on `review-submitted-sub`

9. Go to **"PERMISSIONS"** tab
10. Click **"ADD PRINCIPAL"**
11. **New principals**: Your service account email
12. **Role**: `Pub/Sub Subscriber`
13. Click **"SAVE"**

Screenshots to take:

- Create topic form
- Topic list
- Subscription configuration
- Permissions page

💻 CLI Method

bash

```
# 1. Set variables
export TOPIC_NAME="review-submitted"
export SUBSCRIPTION_NAME="review-submitted-sub"

# 2. Create main topic
gcloud pubsub topics create $TOPIC_NAME \
    --message-retention-duration=7d

# 3. Create dead letter topic
gcloud pubsub topics create review-submitted-dead-letter \
    --message-retention-duration=7d

# 4. Create subscription (temporary - will update later)
gcloud pubsub subscriptions create $SUBSCRIPTION_NAME \
    --topic=$TOPIC_NAME \
    --ack-deadline=600 \
    --message-retention-duration=7d

# 5. Grant publisher permission
gcloud pubsub topics add-iam-policy-binding $TOPIC_NAME \
    --member="serviceAccount:${SA_EMAIL}" \
    --role="roles/pubsub.publisher"

# 6. Grant subscriber permission
gcloud pubsub subscriptions add-iam-policy-binding $SUBSCRIPTION_NAME \
    --member="serviceAccount:${SA_EMAIL}" \
    --role="roles/pubsub.subscriber"

# 7. Verify
gcloud pubsub topics list
gcloud pubsub subscriptions list

gcloud pubsub topics describe $TOPIC_NAME
```

---

6. Firestore Database

🖥️ GUI Method

Step 1: Navigate to Firestore

1. Open navigation menu (☰)
2. Go to **"Firestore"**
3. Click **"SELECT NATIVE MODE"**

Step 2: Create Database

1. **Choose location:**
    - Select **"us-central1 (Iowa)"**
2. Click **"CREATE DATABASE"**
3. Wait for creation (1-2 minutes)
4. You'll see the Firestore Data browser

Step 3: Create Indexes

1. Go to **"Indexes"** tab
2. Click **"CREATE INDEX"**

Index Configuration:

1. **Collection ID**: `analyzed-reviews`
2. **Query scope**: Collection
3. **Fields to index:**
    - Click **"ADD FIELD"**
    - **Field path**: `productId`
    - **Query scopes**: Ascending
    - Click **"ADD FIELD"** again

        - **Field path**: `analyzedAt`
        - **Query scopes**: Descending
    4. Click **"CREATE"**
    5. Wait for index to build (3-5 minutes)
    6. Status will change from "Building" to "Enabled"

Step 4: Grant Permissions

1. Open navigation menu (☰)
2. Go to **"IAM & Admin"** → **"IAM"**
3. Click **"GRANT ACCESS"**
4. **New principals**: Your service account email
5. **Select a role**:
    - Add **"Cloud Datastore User"**
    - Click **"ADD ANOTHER ROLE"**
    - Add **"Cloud Datastore Owner"**
6. Click **"SAVE"**

Screenshots to take:

- Native mode selection
- Database creation
- Index creation form
- Index status page

💻 CLI Method

bash

```
# 1. Create Firestore database
gcloud firestore databases create \
    --location=us-central1 \
    --type=firestore-native

# Wait for creation
echo "Waiting for Firestore database creation..."
sleep 90

# 2. Verify creation
gcloud firestore databases describe --database='(default)'

# 3. Create composite index
gcloud firestore indexes composite create \
    --collection-group=analyzed-reviews \
    --query-scope=COLLECTION \
    --field-config field-path=productId,order=ASCENDING \
    --field-config field-path=analyzedAt,order=DESCENDING

# 4. Check index status (repeat until READY)
gcloud firestore indexes composite list
```

```
# 5. Wait for index to be ready
echo "Waiting for index to build (3-5 minutes)..."
while true; do
    STATUS=$(gcloud firestore indexes composite list --format="value(state)" | head -n 1)
    if [ "$STATUS" = "READY" ]; then
        echo "✓ Index is ready!"
        break
    fi
    echo "Index status: $STATUS - waiting..."
    sleep 30
done

# 6. Grant permissions (already done in service account setup, but verify)
gcloud projects add-iam-policy-binding $PROJECT_ID \
    --member="serviceAccount:${SA_EMAIL}" \
    --role="roles/datastore.user"

gcloud projects add-iam-policy-binding $PROJECT_ID \
    --member="serviceAccount:${SA_EMAIL}" \
    --role="roles/datastore.owner"
```

---

7. Secret Manager

🖥️ GUI Method

Step 1: Get Gemini API Key

1. Open new tab: https://aistudio.google.com/app/apikey
2. Click **"Create API Key"**
3. Select **"Create API key in existing project"**
4. Select your project: `devfest-review-analyzer`
5. Click **"CREATE"**
6. **Copy** the API key (it looks like: `AIzaSy...`)
7. Keep this tab open

Step 2: Navigate to Secret Manager

1. In GCP Console, open navigation menu (☰)
2. Go to **"Security"** → **"Secret Manager"**
3. Click **"CREATE SECRET"**

Step 3: Create Secret

1. **Name**: `gemini-api-key`
2. **Secret value**: Paste your Gemini API key
3. **Regions**: Leave as "Automatic"
4. Click **"CREATE SECRET"**

Step 4: Grant Access

1. Click on your secret `gemini-api-key`
2. Go to **"PERMISSIONS"** tab
3. Click **"GRANT ACCESS"**
4. **New principals**: Your service account email
5. **Role**: `Secret Manager Secret Accessor`
6. Click **"SAVE"**

Screenshots to take:

- Gemini API key creation
- Secret creation form
- Secret details page
- Permissions page

💻 CLI Method

bash

```
# 1. Get Gemini API key manually from:
# https://aistudio.google.com/app/apikey

# 2. Store in environment variable
export GEMINI_API_KEY="YOUR_API_KEY_HERE"

# 3. Create secret
echo -n "$GEMINI_API_KEY" | gcloud secrets create gemini-api-key \
    --data-file=- \
    --replication-policy="automatic"

# 4. Grant service account access
gcloud secrets add-iam-policy-binding gemini-api-key \
    --member="serviceAccount:${SA_EMAIL}" \
    --role="roles/secretmanager.secretAccessor"

# 5. Verify
gcloud secrets list
gcloud secrets versions access latest --secret=gemini-api-key

# 6. Test access (should see your API key)
```

```
gcloud secrets versions access latest --secret=gemini-api-key
--impersonate-service-account=${SA_EMAIL}
```

---

8. Artifact Registry

🖥️ GUI Method

Step 1: Navigate to Artifact Registry

1. Open navigation menu (☰)
2. Go to **"Artifact Registry"** → **"Repositories"**
3. Click **"CREATE REPOSITORY"**

Step 2: Create Repository

1. **Name**: `review-analyzer`
2. **Format**: Docker
3. **Mode**: Standard
4. **Location type**: Region
5. **Region**: `us-central1 (Iowa)`
6. **Description**: `Docker repository for Review Analyzer microservices`
7. **Encryption**: Google-managed encryption key
8. Click **"CREATE"**

Step 3: Note Repository Path After creation, you'll see:

- **Repository path**: `us-central1-docker.pkg.dev/devfest-review-analyzer/review-analyzer`
- Copy this for later use

Screenshots to take:

- Create repository form
- Repository details page

💻 CLI Method

bash

```
# 1. Set variables
export REGISTRY_NAME="review-analyzer"
export REGION="us-central1"

# 2. Create repository
gcloud artifacts repositories create $REGISTRY_NAME \
    --repository-format=docker \
```

```
    --location=$REGION \
    --description="Docker repository for Review Analyzer microservices"

# 3. Configure Docker authentication
gcloud auth configure-docker ${REGION}-docker.pkg.dev

# 4. Set registry path
export REGISTRY="${REGION}-docker.pkg.dev/${PROJECT_ID}/${REGISTRY_NAME}"

# 5. Verify
gcloud artifacts repositories list
gcloud artifacts repositories describe $REGISTRY_NAME --location=$REGION

echo "Registry URL: $REGISTRY"
```

---

9. Cloud Run Deployment

🖥️ GUI Method

Prerequisites:

- All Docker images must be built and pushed to Artifact Registry first
- Complete Phase 10 from the previous guide (Docker Build & Push)

Step 1: Deploy Analytics Service

1. Open navigation menu (☰)
2. Go to **"Cloud Run"**
3. Click **"CREATE SERVICE"**

Configure service:

1. **Container image:**
    - Click **"SELECT"**
    - Navigate to: `us-central1-docker.pkg.dev/[PROJECT_ID]/review-analyzer/analytics:latest`
    - Click **"SELECT"**
2. **Service name**: `analytics`
3. **Region**: `us-central1 (Iowa)`
4. **CPU allocation and pricing:**
    - Select **"CPU is only allocated during request processing"**
5. **Autoscaling:**
    - **Minimum**: 0
    - **Maximum**: 10

6. **Ingress control:**
    - Select **"All"** (we'll configure auth later)
7. **Authentication:**
    - Select **"Allow unauthenticated invocations"** (for now)
8. Click **"CONTAINER, NETWORKING, SECURITY"**

Container settings:

1. **Container port**: `8083`
2. **Resources:**
    - **Memory**: 512 MiB
    - **CPU**: 1
3. **Execution environment:**
    - Select **"First generation"**
4. **Variables & Secrets:**
    - Click **"+ ADD VARIABLE"**
    - **Name**: `PROJECT_ID`
    - **Value**: `devfest-review-analyzer`
5. **Service account:**
    - Select your service account: `review-analyzer-sa@...`
6. Click **"CREATE"**
7. Wait for deployment (1-2 minutes)
8. **Copy the URL** displayed (e.g., `https://analytics-xxx.run.app`)

Repeat for Other Services:

Service 2: Review Ingestion

- Container: `review-ingestion:latest`
- Port: `8081`
- Environment variables:
    - `PROJECT_ID`: your project ID
    - `REVIEW_BUCKET`: your bucket name
    - `PUBSUB_TOPIC`: `review-submitted`

Service 3: AI Analysis

- Container: `ai-analysis:latest`
- Port: `8082`
- CPU: 2
- Memory: 1 GiB
- Timeout: 600 seconds
- Environment variables:
    - `PROJECT_ID`: your project ID
- **Secrets:**
    - Click **"REFERENCE A SECRET"**
    - Secret: `gemini-api-key`

    - Reference method: **"Exposed as environment variable"**
    - Environment variable name: `GEMINI_API_KEY`
    - Version: **"latest"**
- Min instances: 1 (to keep it warm)

Service 4: API Gateway

- Container: `api-gateway:latest`
- Port: `8080`
- Environment variables:
    - `REVIEW_INGESTION_URL`: URL from Review Ingestion service
    - `ANALYTICS_URL`: URL from Analytics service
- Min instances: 1
- Authentication: **"Allow unauthenticated invocations"**

Screenshots to take for each service:

- Service creation form
- Container configuration
- Environment variables
- Service details page after deployment

💻 CLI Method

bash

```
# 1. Set registry URL
export REGISTRY="${REGION}-docker.pkg.dev/${PROJECT_ID}/${REGISTRY_NAME}"

# 2. Deploy Analytics (no dependencies)
gcloud run deploy analytics \
  --image=${REGISTRY}/analytics:latest \
  --platform=managed \
  --region=${REGION} \
  --service-account=${SA_EMAIL} \
  --set-env-vars="PROJECT_ID=${PROJECT_ID}" \
  --allow-unauthenticated \
  --port=8083 \
  --cpu=1 \
  --memory=512Mi \
  --timeout=300 \
  --min-instances=0 \
  --max-instances=10

# Get URL
export ANALYTICS_URL=$(gcloud run services describe analytics --region=${REGION}
--format='value(status.url)')
echo "Analytics URL: $ANALYTICS_URL"
```

```
# 3. Deploy Review Ingestion
gcloud run deploy review-ingestion \
    --image=${REGISTRY}/review-ingestion:latest \
    --platform=managed \
    --region=${REGION} \
    --service-account=${SA_EMAIL} \

--set-env-vars="PROJECT_ID=${PROJECT_ID},REVIEW_BUCKET=${BUCKET_NAME},P
UBSUB_TOPIC=${TOPIC_NAME}" \
    --allow-unauthenticated \
    --port=8081 \
    --cpu=1 \
    --memory=512Mi \
    --timeout=300 \
    --max-instances=20

# Get URL
export REVIEW_INGESTION_URL=$(gcloud run services describe review-ingestion
--region=${REGION} --format='value(status.url)')
echo "Review Ingestion URL: $REVIEW_INGESTION_URL"

# 4. Deploy AI Analysis
gcloud run deploy ai-analysis \
    --image=${REGISTRY}/ai-analysis:latest \
    --platform=managed \
    --region=${REGION} \
    --service-account=${SA_EMAIL} \
    --set-env-vars="PROJECT_ID=${PROJECT_ID}" \
    --set-secrets="GEMINI_API_KEY=gemini-api-key:latest" \
    --allow-unauthenticated \
    --port=8082 \
    --cpu=2 \
    --memory=1Gi \
    --timeout=600 \
    --min-instances=1 \
    --max-instances=10

# Get URL
export AI_ANALYSIS_URL=$(gcloud run services describe ai-analysis --region=${REGION}
--format='value(status.url)')
echo "AI Analysis URL: $AI_ANALYSIS_URL"

# 5. Update Pub/Sub to Push mode
gcloud pubsub subscriptions delete $SUBSCRIPTION_NAME
gcloud pubsub subscriptions create $SUBSCRIPTION_NAME \
    --topic=$TOPIC_NAME \
    --push-endpoint="${AI_ANALYSIS_URL}/process" \
    --push-auth-service-account=${SA_EMAIL} \
```

```
    --ack-deadline=600 \
    --message-retention-duration=7d

# 6. Deploy API Gateway
gcloud run deploy api-gateway \
    --image=${REGISTRY}/api-gateway:latest \
    --platform=managed \
    --region=${REGION} \
    --service-account=${SA_EMAIL} \

--set-env-vars="REVIEW_INGESTION_URL=${REVIEW_INGESTION_URL},ANALYTICS_
URL=${ANALYTICS_URL}" \
    --allow-unauthenticated \
    --port=8080 \
    --cpu=1 \
    --memory=512Mi \
    --timeout=300 \
    --min-instances=1 \
    --max-instances=10

# Get URL
export API_GATEWAY_URL=$(gcloud run services describe api-gateway
--region=${REGION} --format='value(status.url)')
echo "API Gateway URL: $API_GATEWAY_URL"

# 7. Save all URLs
cat > service-urls.txt << EOF
API Gateway: $API_GATEWAY_URL
Review Ingestion: $REVIEW_INGESTION_URL
AI Analysis: $AI_ANALYSIS_URL
Analytics: $ANALYTICS_URL
EOF


cat service-urls.txt
```

---

10. IAM Permissions

🖥️ GUI Method

Configure Public Access (for demo purposes)

For each service that needs to be called by API Gateway:

Review Ingestion:

1. Go to **Cloud Run** → Click **"review-ingestion"**
2. Go to **"PERMISSIONS"** tab
3. Click **"GRANT ACCESS"**
4. **New principals**: `allUsers`
5. **Role**: `Cloud Run Invoker`
6. Click **"SAVE"**
7. Click **"ALLOW PUBLIC ACCESS"** in the confirmation dialog

Analytics:

1. Go to **Cloud Run** → Click **"analytics"**
2. Go to **"PERMISSIONS"** tab
3. Click **"GRANT ACCESS"**
4. **New principals**: `allUsers`
5. **Role**: `Cloud Run Invoker`
6. Click **"SAVE"**

Note: In production, you should use service-to-service authentication instead of public access.

Screenshots to take:

- Permissions page
- Grant access dialog
- Public access confirmation

💻 CLI Method

bash

```
# Grant public access to Review Ingestion
gcloud run services add-iam-policy-binding review-ingestion \
    --region=${REGION} \
    --member="allUsers" \
    --role="roles/run.invoker"

# Grant public access to Analytics
gcloud run services add-iam-policy-binding analytics \
    --region=${REGION} \
    --member="allUsers" \
    --role="roles/run.invoker"

# Verify
gcloud run services get-iam-policy review-ingestion --region=${REGION}
gcloud run services get-iam-policy analytics --region=${REGION}
```

---

11. Monitoring & Logging

🖥️ GUI Method

Step 1: View Logs

1. Open navigation menu (☰)
2. Go to **"Logging"** → **"Logs Explorer"**

3. **Filter logs:**
   - In the query builder, enter:
```
   resource.type="cloud_run_revision"
   resource.labels.service_name="api-gateway"
```
   - Click **"RUN QUERY"**

4. **View different services:**
   - Change `service_name` to: `review-ingestion`, `ai-analysis`, or `analytics`

5. **View errors only:**
```
   resource.type="cloud_run_revision"
   severity>=ERROR
```

Step 2: Create Dashboard

1. Go to **"Monitoring"** → **"Dashboards"**
2. Click **"CREATE DASHBOARD"**
3. **Dashboard name**: `Review Analyzer Metrics`
4. Click **"ADD CHART"**

Chart 1: Request Rate

1. **Chart title**: `Request Rate`
2. **Resource type**: `Cloud Run Revision`
3. **Metric**: `Request count`
4. **Filter**: Add all your services
5. **Aggregation**: `rate`
6. Click **"SAVE"**

Chart 2: Latency

1. Click **"ADD CHART"**
2. **Chart title**: `Request Latency (P95)`

3. **Metric**: `Request latencies`
4. **Aggregation**: `95th percentile`
5. Click **"SAVE"**

Chart 3: Error Rate

1. Click **"ADD CHART"**
2. **Chart title**: `Error Rate`
3. **Metric**: `Request count`
4. **Filter**: `response_code_class = "5xx"`
5. Click **"SAVE"**

Step 3: Create Alerts

1. Go to **"Monitoring"** → **"Alerting"**
2. Click **"CREATE POLICY"**

Alert Configuration:

1. **Select a metric:**
   - **Resource type**: `Cloud Run Revision`
   - **Metric**: `Request count`
   - **Filter**: `response_code_class = "5xx"`
   - **Aggregation**: `rate`
2. **Configure alert trigger:**
   - **Threshold position**: `Above threshold`
   - **Threshold value**: `5` (5% error rate)
   - **For**: `1 minute`
3. **Notification channel:**
   - Add email notification
   - Enter your email
4. **Policy name**: `High Error Rate - Review Analyzer`
5. Click **"CREATE POLICY"**

Screenshots to take:

- Logs Explorer
- Dashboard with charts
- Alert policy configuration

💻 CLI Method

bash

```
# 1. View logs for a specific service
gcloud logging read "resource.type=cloud_run_revision AND
resource.labels.service_name=api-gateway" \
```

```
  --limit=20 \
  --format=json

# 2. View only errors
gcloud logging read "resource.type=cloud_run_revision AND severity>=ERROR" \
  --limit=20

# 3. Create log-based metrics
gcloud logging metrics create review_analyzer_errors \
    --description="Count of errors in review analyzer services" \
    --log-filter='resource.type="cloud_run_revision" AND severity>=ERROR'

gcloud logging metrics create review_analyzer_requests \
    --description="Count of API requests" \
    --log-filter='resource.type="cloud_run_revision" AND httpRequest.requestUrl=~".*"'

# 4. List metrics
gcloud logging metrics list

# 5. For dashboards and complex alerts, use the GUI

# Or create a dashboard JSON file and import it
```

Testing Your Deployment

🖥️ GUI Method

Using Cloud Console:

1. Go to **Cloud Run** → **api-gateway**
2. Copy the URL
3. Open the URL in browser (you'll see the API info)
4. Use a REST client (Postman, Insomnia, or Thunder Client in VS Code)

Test with Postman:

1. Create new request
2. Method: POST
3. URL: `https://[YOUR-API-GATEWAY-URL]/api/reviews`
4. Headers: `Content-Type: application/json`
5. Body (JSON):

```
json
{
  "productId": "test-product-001",
  "rating": 5,
```

```
  "reviewText": "This product is absolutely amazing! Exceeded all expectations.",
  "reviewerName": "Test User"
}
```

6. Click **Send**

View Results:

1. Wait 15 seconds
2. Create GET request
3. URL: `https://[YOUR-API-GATEWAY-URL]/api/analytics/test-product-001`
4. Click **Send**

💻 CLI Method

bash

```
# 1. Test health endpoints
echo "Testing API Gateway health..."
curl -s ${API_GATEWAY_URL}/health | jq .

# 2. Submit a review
echo "Submitting test review..."
curl -X POST ${API_GATEWAY_URL}/api/reviews \
  -H "Content-Type: application/json" \
  -d '{
    "productId": "test-product-001",
    "rating": 5,
    "reviewText": "This product is absolutely amazing! Exceeded all expectations.",
    "reviewerName": "Test User"
  }' | jq .

# 3. Wait for processing
echo "Waiting 15 seconds for AI analysis..."
sleep 15

# 4. Get analytics
echo "Fetching analytics..."
curl -s ${API_GATEWAY_URL}/api/analytics/test-product-001 | jq .

# 5. Check Firestore
ACCESS_TOKEN=$(gcloud auth print-access-token)
curl -s
"https://firestore.googleapis.com/v1/projects/${PROJECT_ID}/databases/(default)/documents/analyzed-reviews" \
  -H "Authorization: Bearer $ACCESS_TOKEN" | jq '.documents[0] | {reviewId: .name, sentiment: .fields.sentiment.stringValue}'
```

```
# 6. Check Cloud Storage
gsutil ls -r gs://${BUCKET_NAME}/reviews/
```

Quick Reference Card

Important URLs

Resource	GUI Navigation	URL Pattern
Console Home	-	https://console.cloud.google.com
Cloud Run	☰ → Cloud Run	https://console.cloud.google.com/run
Firestore	☰ → Firestore	https://console.cloud.google.com/firestore
Cloud Storage	☰ → Cloud Storage	https://console.cloud.google.com/storage
Pub/Sub	☰ → Pub/Sub	https://console.cloud.google.com/cloudpubsub
Logs Explorer	☰ → Logging	https://console.cloud.google.com/logs
Monitoring	☰ → Monitoring	https://console.cloud.google.com/monitoring
IAM	☰ → IAM & Admin	https://console.cloud.google.com/iam-admin
Secret Manager	☰ → Security → Secret Manager	https://console.cloud.google.com/security/secret-manager

CLI Quick Commands

bash

```
# Set project
gcloud config set project PROJECT_ID

# View all services
gcloud run services list --region=us-central1

# View logs
```

```
gcloud logging read "resource.type=cloud_run_revision" --limit=50

# Redeploy service
gcloud run deploy SERVICE_NAME --image=IMAGE_URL --region=us-central1

# Check Firestore
gcloud firestore databases describe --database='(default)'

# List buckets
gsutil ls

# View Pub/Sub topics
gcloud pubsub topics list
```